数据结构基础 (FDS) 期中核心复习讲义

根据 FDS 课程 PPT 整理

2026 年 4 月 23 日

目录

1 第一部分：算法分析基础 (Algorithm Analysis) 2

1.1 1. 算法的定义与五大特征 2

1.2 2. 复杂度渐近表示法 2

1.3 3. 一般法则与最大子列和问题 2

2 第二部分：线性表、栈与队列 (Lists, Stacks, Queues) 3

2.1 1. 线性表 (List ADT) 3

2.2 2. 栈 (Stack ADT) 3

2.3 3. 队列 (Queue ADT) 3

3 第三部分：树与二叉树 (Trees & Binary Trees) 4

3.1 1. 基本术语 4

3.2 2. 二叉树的性质 4

3.3 3. 线索二叉树 (Threaded Binary Trees) 4

3.4 4. 二叉搜索树 (Binary Search Trees, BST) 4

4 第四部分：优先队列 (Priority Queues / Heaps) 5

4.1 1. 结构与堆序性质 5

4.2 2. 核心操作 5

5 第五部分：并查集 (The Disjoint Set ADT) 6

5.1 1. 数据结构表示 6

5.2 2. 优化策略 6

6 第六部分：线段树 (Segment Tree) 7

6.1 1. 动机与结构 7

6.2 2. 核心操作与复杂度 7

1 第一部分：算法分析基础 (Algorithm Analysis)

1.1 1. 算法的定义与五大特征

算法是为了完成特定任务而规定的一组有限指令集。必须满足以下五个准则：

- **输入 (Input)**: 外部提供的零个或多个量。
- **输出 (Output)**: 至少产生一个量。
- **确定性 (Definiteness)**: 每条指令清晰无歧义。
- **有限性 (Finiteness)**: 算法在执行有限步后必须终止。
- **有效性 (Effectiveness)**: 每条指令足够基本，原则上可用纸笔计算完成。

注：*程序 (Program)* 可以不满足有限性（例如操作系统）。

1.2 2. 复杂度渐近表示法

- $T(N) = O(f(N))$: 存在正常数 c 和 n_0，使得对于所有 $N \geq n_0$，有 $T(N) \leq c \cdot f(N)$。(表示复杂度上界)
- $T(N) = \Omega(g(N))$: 表示复杂度下界。
- $T(N) = \Theta(h(N))$: 当且仅当 $T(N) = O(h(N))$ 且 $T(N) = \Omega(h(N))$。(确界)
- $T(N) = o(p(N))$: $T(N) = O(p(N))$ 且 $T(N) \neq \Theta(p(N))$。(严格上界)

1.3 3. 一般法则与最大子列和问题

- **For 循环**: 运行时间最多是循环内部语句运行时间乘以迭代次数。
- **嵌套循环**: 由内而外分析，语句运行时间乘以所有循环大小的乘积。
- **连续语句**: 直接相加，即取最大值 $O(\max(f(N), g(N)))$。
- **If/Else**: 运行时间不超过判断语句加上分支中运行时间较长者的和。

最大子列和问题 (Max Subsequence Sum):

- 算法 1 (暴力枚举起点终点及求和): $O(N^3)$
- 算法 2 (优化求和): $O(N^2)$
- 算法 3 (分治法 Divide and Conquer): $O(N \log N)$
- 算法 4 (在线处理 Online Algorithm): 扫描一次，当前和 < 0 时直接抛弃并归零重新开始。时间复杂度 $O(N)$。

二分查找 (Binary Search): 对于已排序的静态数据，最坏时间复杂度为 $O(\log N)$。

2 第二部分：线性表、栈与队列 (Lists, Stacks, Queues)

2.1 1. 线性表 (List ADT)

- **数组实现 (Array)**: 连续内存分配。查找第 K 个元素时间为 $O(1)$，但插入和删除涉及大量数据移动，时间复杂度 $O(N)$。必须预估最大容量。
- **链表实现 (Linked List)**: 离散内存，通过指针连接。插入和删除只需修改指针，时间 $O(1)$（已知位置的前提下）。查找需要遍历，时间 $O(N)$。
- **游标实现 (Cursor Implementation)**: 针对不支持指针的语言。使用数组和下标模拟链表与内存池 (freelist)。

2.2 2. 栈 (Stack ADT)

后进先出 (LIFO, Last-In-First-Out)。插入和删除只在栈顶 (Top) 进行。

- **链表实现**: 在链表头部进行压栈 (Push) 和出栈 (Pop)，时间均为 $O(1)$。
- **数组实现**: 使用变量记录 `TopOfStack`。Push 时先加后赋值，Pop 时先取值后减。若栈为空时 Pop 为 ADT 错误，满时 Push 为实现限制错误。

2.3 3. 队列 (Queue ADT)

先进先出 (FIFO, First-In-First-Out)。在队尾 (Rear) 插入，在队头 (Front) 删除。

- **循环数组实现 (Circular Array)**: 解决“假溢出”问题。队头队尾指针在到达数组末尾时通过求余操作回到起始位置。
- **判空与判满**: 由于满和空时指针可能重合，解决方法有：(1) 维护一个 `Size` 计数器；(2) 数组多预留一个空位，不存元素。

3 第三部分：树与二叉树 (Trees & Binary Trees)

3.1 1. 基本术语

- 节点度数 (Degree): 节点的子树个数。叶节点度数为 0。
- 深度 (Depth): 从根到该节点的唯一路径长度（根的深度为 0）。
- 高度 (Height): 从该节点到最深叶节点的路径长（叶子的高度为 0）。树的高度等于根的高度。

3.2 2. 二叉树的性质

- 第 i 层的最大节点数为 2^{i-1} ($i \geq 1$)。
- 深度为 k 的二叉树最大节点总数为 $2^k - 1$ ($k \geq 1$)。
- 对于任意非空二叉树，度数为 0 的叶节点数 n_0 和度数为 2 的节点数 n_2 满足: $\mathbf{n_0 = n_2 + 1}$。

3.3 3. 线索二叉树 (Threaded Binary Trees)

在一棵有 N 个节点的二叉树中，存在 $N + 1$ 个空指针。为了充分利用并加速遍历：

- **Rule 1**: 如果 `Tree->Left` 为空，则用线索指向该节点在中序遍历下的**前驱** (predecessor)。
- **Rule 2**: 如果 `Tree->Right` 为空，则用线索指向该节点在中序遍历下的**后继** (successor)。

3.4 4. 二叉搜索树 (Binary Search Trees, BST)

性质：对任意节点，左子树所有节点的值 < 该节点的值 < 右子树所有节点的值。

- **查找/插入**: 平均 $O(\log N)$，最坏情况（如单向偏斜）$O(N)$。
- **删除 (Deletion)**:
  1. 叶节点：直接删除。
  2. 单孩节点：用其唯一孩子顶替它。
  3. 双孩节点：用右子树中的**最小节点**（或左子树最大节点）的值替换它，然后递归删除那个最小节点。
- **懒惰删除 (Lazy Deletion)**: 给节点打上“已删除”标记而不是立刻释放内存，在删除操作不频繁时提高效率。

4 第四部分：优先队列 (Priority Queues / Heaps)

优先队列用于快速获取优先级最高/最低的元素（以最小堆 Min-Heap 为例）。

4.1 1. 结构与堆序性质

- **结构性质 (完全二叉树)**: 除了底层，其他层全满，底层从左到右填充。
- **数组表示**: 根存储在下标 1。对下标 i 的节点：左孩子在 $2i$，右孩子在 $2i+1$，父节点在 $\lfloor i/2 \rfloor$。
- **堆序性质 (Heap-Order)**: 任何节点的值必须 $\leq$ 其所有子节点的值。

4.2 2. 核心操作

- **插入 (Insert)**: 将新元素放在数组最后，然后执行**上滤 (Percolate Up)**，与父节点不断比较并交换，直至满足堆序性质。时间复杂度 $O(\log N)$。
- **删除最小元 (DeleteMin)**: 移走根节点，将最后一个元素移到根位置，然后执行**下滤 (Percolate Down)**，与两个孩子中较小的一个比较并交换。时间复杂度 $O(\log N)$。
- **建堆 (BuildHeap)**: 给定 N 个元素的乱序数组，从最后一个非叶子节点（下标 $\lfloor N/2 \rfloor$）开始，逐个向前执行下滤操作。时间复杂度为 $\mathbf{O(N)}$。

5 第五部分：并查集 (The Disjoint Set ADT)

用于处理动态等价关系（如网络连通性）。核心操作是 `Find` 和 `Union`。

5.1 1. 数据结构表示

使用数组 `S[N]` 存储一棵棵树构成的森林。`S[i]` 存储节点 i 的父节点索引。对于根节点，`S[i]` 通常存储一棵树的高度或大小的负值。

5.2 2. 优化策略

- **按秩/按高合并 (Union-by-Rank/Height)**：合并时，总是将较浅的树作为较深树的子树，保证树的高度不过快增长。
- **路径压缩 (Path Compression)**：在执行 `Find(X)` 时，将查找路径上访问到的所有节点直接连到根节点上。
- 注意：路径压缩会改变树高，因此“按高合并”中的“高”退化为一种“秩 (Rank)”的粗略估计，这两种优化可以并存。

时间复杂度：对于 M 次操作和 N 个元素，最坏时间复杂度为 $O(M\alpha(M,N))$。其中 α 是反阿克曼函数，增长极其缓慢，实际可视为常数。

6 第六部分：线段树 (Segment Tree)

6.1 1. 动机与结构

用于解决频繁的数组区间查询（如区间求和）和单点更新问题。

- 普通数组查询 $O(N)$ 较慢，线段树可将其优化至 $O(\log N)$。
- **结构**: 也是一棵二叉树。叶子节点存储数组原元素，内部节点存储其对应区间的汇总信息。
- 如果用数组存储线段树，对长度为 N 的数组，通常需要开辟大小约为 **4N** 的数组。

6.2 2. 核心操作与复杂度

- **建树 (Build)**: 递归建立左子树和右子树，合并区间和。复杂度 $O(N)$。
- **更新 (Update)**: $O(\log N)$。自顶向下查找到叶子节点并更新，然后自底向上更新沿途的所有父节点区间。
- **查询 (Query)**: $O(\log N)$。递归将目标区间不断拆解为树所维护的标准区间片段并返回汇总值。